Prompt Injection Detection for Email Agents Through Attack Chain Modeling

Ahmad Hashmi, Dhyey Patel, Yunting Yin
Eastern Michigan University

Introduction

Email assistants help users summarize messages, draft replies, and manage their inboxes. To perform these tasks, they must read emails from potentially untrusted sources. An attacker can embed instructions in a seemingly ordinary message.

A single **attack** or **benign** label does not explain how an injection progresses or where it fails. We examine prompt injection as an **attack chain**: the malicious message is retrieved, bypasses defenses, triggers a tool call, and supplies the attacker's intended recipient and content. Distinguishing these stages helps reveal which behaviors are predictable and which remain difficult to detect.

Datasets

LLMail-Inject provides 461,640 attack attempts with stage outcomes. We complement it with the six datasets below to evaluate detection across different settings. NotInject supplies benign examples that resemble attacks for threshold calibration and false positive evaluation.

Table 1. Dataset size and class distribution.

Dataset	Rows	Attack	Benign
BIPIA generated binary	176,200	175,000	1,200
NotInject	339	0	339
NVIDIA Agentic IPI	1,272	1,272	0
PromptShield	43,425	16,441	26,984
ShieldLM	54,162	18,965	35,197
Neuralchemy	6,274	3,736	2,538

Where do email injection attacks fail?

Most LLMail-Inject attacks stop before achieving their goal. 49.7% are retrieved but blocked, while 46.8% bypass defenses without triggering a tool call. Only **0.7%** achieve **full success**. Analyzing individual stages reveals where attacks fail and motivates our detection framework.

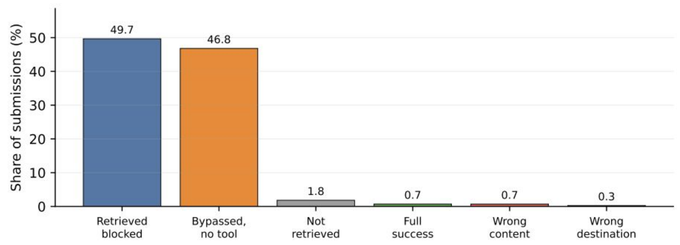


Figure 1. Attack outcomes in LLMail-Inject

Evaluation

We use random and phase transfer splits, component ablations, and comparisons with published detectors and a zero-shot OpenAI LLM judge. Metrics include F1, ROC-AUC, recall, and false positive rate. Threshold calibration and testing use separate NotInject subsets across three rotations.

Proposed Framework

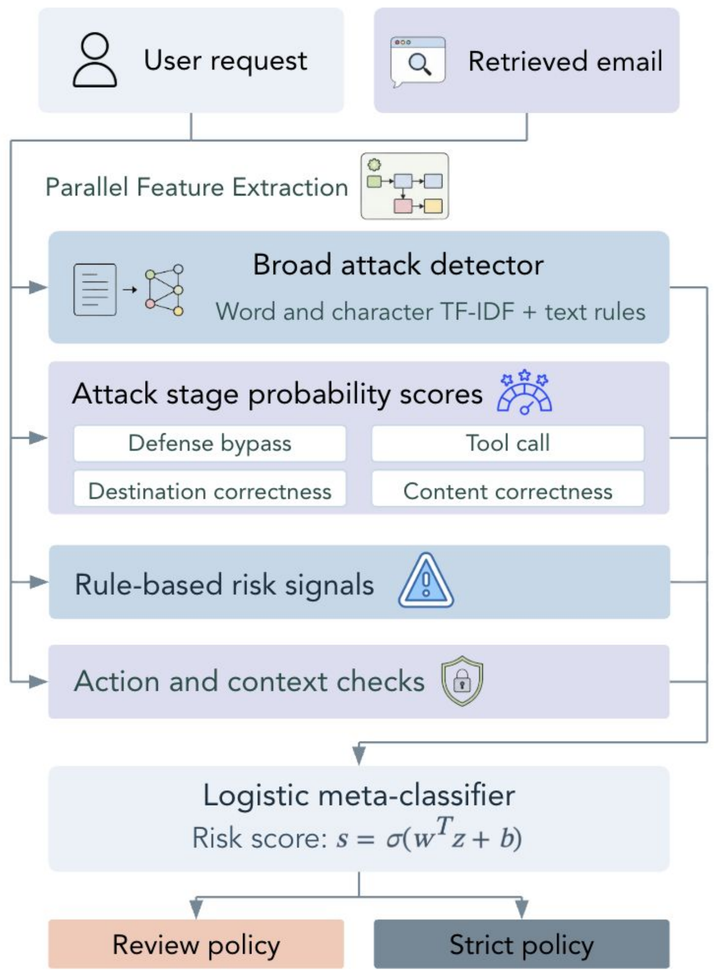


Figure 2. Proposed framework workflow.

Our framework combines an **attack detector**, **stage verifiers**, along with **action and context checks**.

The stage verifiers identify patterns associated with defense bypass, tool use, and attack completion, while contextual checks flag potentially unauthorized actions.

A logistic regression meta policy learns to combine these signals using held-out validation scores.

The resulting risk score supports two operating modes:

1. review-oriented detection
2. stricter screening calibrated with NotInject benign examples to limit false alarms

Which Components Help?

Table 2. Component ablation.

Variant	Policy	Mean F1	NotInject FPR	NVIDIA Rec.	BIPIA F1
Selective stages	review	0.643 ± 0.002	27.73%	100.0%	0.745
All stages	review	0.643 ± 0.003	27.14%	100.0%	0.744
Text/rules only	strict	0.281 ± 0.149	0.88%	4.01%	0.182
All stages	strict	0.348 ± 0.096	1.77%	12.40%	0.183
Action/context only	strict	0.386 ± 0.024	0.88%	2.33%	0.735
Selective stages + action/context	strict	0.400 ± 0.057	2.36%	2.33%	0.733
All stages + action/context	**strict**	$\mathbf{0.406 \pm 0.057}$	2.06%	2.33%	0.732
Nonlinear stacker	strict	0.347 ± 0.034	1.47%	21.72%	0.785

Combining all stages with action and context features achieves the **highest strict mode mean F1 (0.406)**, compared with 0.281 for text rules alone. Action and context features contribute substantially, reaching 0.386 on their own. The selective stage combination performs similarly (0.400), so the benefit of retaining every stage is modest. Review mode achieves higher F1 but produces roughly 27% false positives, highlighting the **trade-off** between detecting attacks and avoiding false alarms.

Results

Table 3. Hard negative results.

Train	Test	F1 / FPR	AUC
LLMail	BIPIA	0.453	0.572
BIPIA	BIPIA	0.738	0.904
BIPIA	LLMail	0.747	0.780
LLMail+BIPIA	BIPIA	0.770	0.899
LLMail+BIPIA	NotInject	65.2% FPR	–
LLMail+BIPIA+NotInject	**NotInject**	**0.6% FPR**	**–**
LLMail+BIPIA+NotInject	**BIPIA**	**0.730**	**0.904**
LLMail+BIPIA+NotInject	**LLMail**	**0.995**	**1.000**

In the initial transfer experiment, adding NotInject **reduces benign false positives** from 65.2% to 0.6%, while BIPIA F1 decreases modestly from 0.770 to 0.730. This highlights the importance of **learning from legitimate content that resembles an attack**, helping the detector distinguish suspicious wording from malicious intent. Under matched strict calibration, the final framework achieves **a mean F1 of 0.406**, exceeding all five published detector baselines (best: 0.216), with a 2.06% NotInject false positive rate. It also achieves substantially higher BIPIA F1, supporting its effectiveness under low false positive calibration. However, NVIDIA attack recall remains low at 2.33%, showing that this operating policy sacrifices attack coverage.

Table 4. Matched strict transfer comparison.

Method	Mean F1	NotInject FP / 339	NVIDIA Rec.	BIPIA F1
ProtectAI v2	0.216	13 (3.83%)	0.00%	0.000
TestSavant tiny	0.130	8 (2.36%)	0.00%	0.000
ProtectAI v1	0.114	3 (0.88%)	0.00%	0.000
Arkaean DistilBERT	0.094	5 (1.47%)	0.00%	0.000
deepset DeBERTa	0.014	6 (1.77%)	0.00%	0.013
Proposed, all-stage action/context	**0.406**	7 (2.06%)	**2.33%**	**0.732**

Comparison with an LLM Judge

Table 5. Comparison with a zero-shot OpenAI LLM judge.

Method	NotInject FPR	NVIDIA Rec.	BIPIA F1	LLMail F1
Framework, review[a]	27.73%	100.00%	0.745	0.665
Framework, strict[b]	2.06%	2.33%	0.732	0.665
OpenAI judge, default	1.47%	75.16%	0.630	0.919
OpenAI judge, strict	1.77%	75.16%	0.625	0.891
OpenAI judge, ultra-strict	1.18%	73.45%	0.565	0.810

[a] Selective stages without action and context, using the validation-F1 threshold. [b] All stages plus action and context.

We compare our framework with a zero-shot GPT-4.1-mini judge on the same held-out examples. Under strict calibration, our framework achieves higher BIPIA F1 (0.732 vs. 0.625), while the judge achieves higher LLMail F1 and NVIDIA recall, with slightly fewer false positives. The framework's review configuration detects all NVIDIA attacks but raises the benign false positive rate to 27.73%. These results reveal that our framework supports local detection without per-message LLM calls, but the judge generalizes better on several benchmarks.

Takeaway and Next Steps

Attack chain modeling with contextual signals and hard negative training improve detection. Our framework achieves higher strict policy mean F1 than the evaluated published detectors. Next, we plan to evaluate it in an email agent.

Code: github.com/AcaiLab/Email

This work was supported by the Honors Undergraduate Research Fellowship at Eastern Michigan University.